

VULNERABILITY RESEARCH REPORT

ROOT ON LINUX. DEAD ON ANDROID.

VERDICT: DOS-ONLY (0/102K RUNS)



```
+-----+
| TARGET : CVE-2026-46242 ("Bad Epoll" - fs/eventpoll.c UAF) |
| DISCOV : Jaeyoung Chung (Google kernelCTF)                |
| TESTED  : x86_64 Linux VM vs ARM64 Android GKI 6.1 (testbed) |
| RESULT  : 102,740 Race Runs -> 0 Hits -> 21 Dead Ends Killed |
+-----+
```

The Concurrency Flaw



THREAD A: epoll_ctl(EPOLL_CTL_DEL)

THREAD B: ep_free() / teardown

- 1. ep_remove() called
- 2. unlinks epitem from rbtree
- 3. writes NULL to f_ep_links
- [RACE WINDOW: ~125ns]
- 4. writes to FREED MEMORY (UAF!) <===== RACE COLLISION POINT

- 1. ep_free() loops item list
- 2. frees epitem -> kmalloc-192
- 3. SLUB marks chunk FREE

99%

X86_64 VM RELIABILITY

0 / 102,740

ARM64 GKI RACE HITS

How Root Was Won on x86

- **01. Heap Reclamation (msg_msg)**

Sprayed msg_msg queues in kmalloc-192 to reclaim the dangling epitem immediately upon free.

- **02. KASLR Leak via OOB Read**

Corrupted m_ts field allowed reading uninitialized kernel heap memory, leaking kernel text base.

- **03. ROP Payload & Privilege Escalation**

Stack pivot gadget executed commit_creds(&init_cred), popping clean root shell (UID 0).

```
# id
uid=0(root) gid=0(root) groups=0(root)
[+] Exploit reliability verified: 99/100 runs succeeded.
```

The 125ns Timing Wall



EXECUTION WINDOW MEASUREMENT (GKI 6.1 ARM64):

__ep_remove() instruction count: ~250 - 550 CPU cycles
At 2.0 GHz clock frequency: ~125 - 275 nanoseconds
cond_resched() state: NO-OP (Voluntary Preemption)

RESULT: Thread B can NEVER interrupt Thread A in this window
without artificial hardware breakpoints or GDB traps.

0 Hits in 102k Runs

GDB watchpoints created a false illusion of exploitability that completely vanished during raw un-halted kernel execution.

4 Escalation Paths. All Dead.

- **Chain 0: Refcount Decrement (EXP-004)**

percpu_counter_dec always decremented the valid outer epoll, not the freed inner structure.

- **Chain 1: Dual-Watch KASLR Leak (EXP-008)**

Mathematically impossible: multi-epitem pointer writes and single-epitem UAFs are mutually exclusive.

- **Chain 2 & 3: Arbitrary Decrement -> LPE (EXP-012)**

Reclaiming with msg_msg locked decrement onto root_user, leaving only a fixed NULL write at offset 160.

Offset 160 Corruption Audit



STRUCT	OFFSET 160 FIELD	OBSERVED RUNTIME BEHAVIOR
-----	-----	-----
fib6_info	fib6_metrics pointer	[PANIC] NULL route deref
snd_timer_user	queue list_head	[PANIC] Corrupt list walk
packet_fanout	next member pointer	[PANIC] Teardown null deref

Audit Conclusion:

There is zero reachable kmalloc-192 structure in the GKI tree where zeroing 8 bytes at offset 160 results in anything other than an immediate kernel panic.

Why Mobile Hardening Won

- [+] PAC (Pointer Auth)
--> Neutralized function pointer forgery & return address hijacking.
- [+] kCFI & BTI (Control Flow Integrity)
--> Enforced forward-edge indirect branch validation across kernel.
- [+] Slab Isolation & Memory Latency
--> Physical ARM64 bus contention & SLUB freelist hardening broke heap grooming determinism.

Modern Android mitigations performed as designed. Even with a high-severity primitive, defense-in-depth reduced a 99% root exploit to a local DoS.

The Complete 26-Page Post-Mortem

INCLUDED IN FULL TECHNICAL REPORT:

- * Full Verification Ledger (VER-001 through VER-039)
- * 21 Dead Ends documented with killing disassembly & traces
- * Tier 1 x86_64 ROP gadget chains & offset tables
- * Android GKI synthetic build & debugging harness

WANT THE 26-PAGE PDF?

Drop **"EPOLL"** in the LinkedIn comments and I'll send the direct report straight to your DMs.